

Hollow Soul - Unity

M0 to M11 Implementation Audit and Next Phase Milestones

Audit date: April 26, 2026. Repository audited: /Users/martinjedrzejewski/Documents/GitHub/Unity/Hollow Soul - Unity

Executive Finding

The M0-M11 sequence is successful as a prototype-lock implementation. The project now has a working shared Unity runtime slice, imported room gameplay, combat, deterministic branch traversal, run economy, persistence, Room Designer export/playtest, content pipeline, platform polish profiles, and a prototype-lock validation layer. The implementation is not yet a production-ready game or full macro-room/procedural branch system, but it is a solid and validated foundation for the next phase.

Evidence	Audit Result
EditMode tests	65 total, 65 passed, 0 failed
Milestone validators	M0 through M11 present; M0-M10 passed after M11; M11 passed
Generated scenes	6: Boot, MainMenu, Game_Windows, Game_VisionOS_Bounded, Game_VisionOS_Immersive, RoomDesigner
Milestone docs	12 markdown files: M0 through M11
Script coverage	153 C# files across runtime/editor assemblies, including 26 editor generation/validation files
Prototype content	31 materials, 10 VFX cue assets, 10 audio cue assets, 3 platform polish profiles
Prototype lock assets	Checklist, performance budget, and build handoff assets generated under Assets/_Hollow/Data/PrototypeLock

Overall Status

This audit separates prototype success from long-term plan completion. Most milestones pass against their milestone scope. A few items from the wider Unity architecture plan were intentionally simplified or deferred, especially CI/build automation, explicit macro-room topology, final content import, and production Vision Pro packaging.

Milestone	Original Intent	Status	Audit Notes / Differences
M0	Unity foundation, packages, asmdefs, URP/Linear setup, input, Addressables, CI/build profiles.	Prototype pass	Assets/_Hollow root exists with assembly split, scenes, tests, docs, and validation. Addressables arrived in M9; build scenes are now configured. CI/build automation and a dedicated .inputactions asset are not yet present.
M1	Boot flow, main menu, profile slots, platform route selection, Windows and Vision Pro shell routes.	Pass	Main menu, profile store, slot cards, route model, platform services, generated scenes, and tests are present.
M2	Shared runtime shell, player spawn, camera rigs, tabletop scale, bounded/immersive presentation split.	Pass	WorldPresentationRoot scale policy, camera rigs, shell canvas separation, placeholder room/player, and platform scenes are implemented.
M3	HollowRuntime V2 import, default sample room, floor/doors/obstacles/spawns presentation.	Pass	V2 importer, serializable runtime room models, 13x7 sample JSON, static builders, and tests exist.
M4	First playable combat loop: movement, shooting, enemies, room clear, HUD.	Pass	Player movement, local-space collision, projectiles, chaser enemies, health/damage, objective clear, and HUD are covered.

Milestone	Original Intent	Status	Audit Notes / Differences
M5	Enemy archetype framework, boss shell, spawn weighting, difficulty tiers, readability diagnostics.	Prototype pass	Normal/flying/fast/heavy definitions, catalog, difficulty tuning, diagnostics, and boss shell exist. Full boss behaviors and broad encounter content remain later work.
M6	Branch graph, traversal, minimap, rewards, hub return.	Prototype pass	Deterministic five-room cross, door gating, traversal, minimap model/controller, runtime rewards, and hub portal exist. Procedural graph generation is intentionally deferred.
M7	Run economy, active-run persistence, meta progression, Continue/New Run flow, transient safety.	Pass	Run snapshots, profile schema v2, deterministic reward grants, stat effects, banking on completion, death clear, and transient guard are implemented.
M8	Unity Room Designer mode, 1m grid, tools, JSON/USDA export, transient playtest.	Prototype pass	RoomDesigner scene/prefab, project/store/compiler/exporters/input/controller and tests exist. V1 is 1x1 only; share sheet and macro presets are deferred.
M9	Content pipeline: materials, VFX/audio cues, Addressables, import validation.	Pass	31 prototype material roles, 10 VFX cues, 10 audio cues, content catalog, resolver/presenters, Addressables labels, and validator exist.
M10	Platform polish for Windows, Vision Pro bounded tabletop, and immersive comfort/performance pass.	Pass	Three platform polish profiles, profile applier, comfort vignette metadata, camera rig/scene wiring, and validator exist.
M11	Prototype lock: QA checklist, performance budgets, save/load coverage, content validation, build handoff.	Pass	PrototypeLock checklist, performance budget, build handoff assets, validation harness, M11 validator, docs, and tests exist.

File Structure Audit

The current structure follows the planned Assets/_Hollow ownership model and has grown in the expected direction. The most important differences are not defects; they are signs of a prototype-first implementation that now needs a scale-up pass before heavy content production.

Type	Finding
Strong alignment	Canonical root is Assets/_Hollow with Art, Audio, Data, Prefabs, Scenes, Scripts, Tests, Resources, and Docs. Script assemblies largely match the architecture plan.
Implemented differently	Addressables data is generated under Assets/AddressableAssetsData, which is Unity-standard rather than under Assets/_Hollow. This is expected.
Implemented differently	Player movement/weapon runtime lives in Hollow.Combat, while Hollow.Entities currently contains PlaceholderPlayerController and PlayerSpawnPoint. This is fine for prototype but should be reorganized before large content growth.
Implemented differently	Diagnostics are split between Hollow.Combat diagnostics and Hollow.Editor validation. Hollow.Diagnostics itself remains minimal.
Partial	No dedicated HollowControls.inputactions asset is visible. Input is currently captured through custom gameplay/designer input readers and snapshots.
Partial	DeveloperSandbox.unity from the larger Unity handoff plan is not present. Current sandbox-like functionality is covered by RoomDesigner and generated game scenes.
Deferred	Macro-room structures such as BranchRoomInstanceId, RoomInstanceFootprint, and explicit imported-room BranchConnection are not fully implemented yet. Current branch code uses BranchRoomId and deterministic connections.
Deferred	Final content folders exist as placeholders, but production models, textures, music, SFX, remote Addressables, and platform certification packaging are not in scope yet.

Script Assembly Footprint

Assembly / Area	C# files
Core	9
Platform	8
Input	3
UI	10
World	3
Branches	9
Rooms	10
RoomDesigner	9
Combat	28
Entities	2
Rewards	13
Persistence	11
Presentation	11
Diagnostics	1
Editor	26

Interpretation: the implementation is cleanly segmented by assembly, but the Editor generator layer is intentionally large because it creates scenes, prefabs, data, Addressables labels, and validators. Before content production accelerates, the

generator layer should be decomposed into smaller helpers.

Runtime Model Audit

Model Area	Audit Finding
Room runtime	HollowRuntime V2 importer and room models are implemented for the 13x7 sample and Room Designer export. Multi-footprint room semantics remain next-phase work.
Branch runtime	Current branch model is deterministic and directional. It validates traversal, rewards, minimap, and save state, but it is not yet the explicit port-to-port macro-room graph described in the long-term plan.
Economy	Run economy is deterministic and intentionally small: four room rewards, souls, and passive stat effects. Random pools, shops, cards, and item choice UI are deferred.
Persistence	Profile schema v2 stores active run snapshots, banked souls, completed runs, and transient guard behavior. It is suitable for prototype but needs backup/corruption/migration hardening.
Platform	Windows, bounded tabletop, and immersive routes share gameplay and use platform polish profiles. Production PolySpatial packaging, performance capture, and device QA are not complete yet.
Designer	Room Designer has compiler, store, JSON and USDA exporters, keyboard/controller-style input snapshots, and transient playtest. It remains a 1x1 graybox authoring tool.

Validation And Test Coverage

The strongest part of the current implementation is the validation rhythm. Each milestone has an asset generator, validator, doc, and EditMode tests. M11 composes earlier validation into a prototype-lock gate and adds a temp-store save/load exercise so persistence is not merely compile-checked.

Coverage Area	Current State	Remaining Gap
Milestone validators	M0-M11 validators exist; M0-M10 passed after M11; M11 passed.	Add one aggregate command/menu item that runs all validators and writes a machine-readable report.
EditMode tests	65/65 passed in latest run.	Add PlayMode smoke tests once runtime scenes are stable enough.
Save/load	JsonProfileStore temp validation covers new run, checkpoint, completion, and death clear.	Add migration/corruption/backups and longer branch snapshots.
Content validation	M9/M11 validate materials, cue catalog, Addressables labels, prefab refs, and naming.	Add production import validators for models, textures, audio, LFS, and Addressables build artifacts.
Platform validation	M10/M11 validate profiles, world scale, HUD separation, and comfort metadata.	Add real build/device performance capture for Windows and Vision Pro.

Risk Register

Risk	Description
High	Macro rooms and explicit door-port traversal are the next architectural pressure point. Do this before adding many authored rooms.
High	Build automation is still lighter than the milestone plan implies. A prototype lock is valuable, but the next phase needs one-command build and validation.
Medium	Generated asset code is effective but large. Milestone generators should eventually be split into smaller scene/prefab/data builder helpers.
Medium	The player/enemy/content architecture is good for prototype, but content scale will require clearer separation between definitions, runtime state, controllers, and presentation.
Medium	Vision Pro support currently has presentation profiles and scene routes, but not full production packaging/performance certification.
Low	Git status includes many untracked generated M9-M11 assets. This is expected during implementation but should be staged/reviewed as one coherent prototype-lock changeset.

Recommended Next Phase Milestones

Recommended sequencing: first make the prototype buildable and repeatable, then scale the room/branch architecture, then add gameplay content and final platform polish. This keeps the project from piling content on top of a temporary five-room graph.

Milestone	Goal	Implementation Focus	Acceptance Criteria
M12	Build and Release Automation	Create repeatable Windows and visionOS build profiles, local build scripts, CI-style validation runner, artifact naming, build number stamping, and release notes generation.	A clean build command produces Windows prototype artifacts and a validation report. M0-M11 validators and EditMode tests run from one command.
M13	Room Runtime Scale-Up and Macro-Room Import	Expand HollowRuntime V2 and Room Designer from 1x1 into 2x1, 1x2, 2x2, and L rooms with explicit door ports, footprints, sidecar room catalogs, and import fixtures.	Imported macro rooms remain one logical room instance, preserve all same-side ports, and playtest without breaking legacy single rooms.
M14	Procedural Branch Generation V1	Replace the deterministic five-room cross with seeded branch generation using room catalogs, room weights, explicit door-port matching, basic locks/secrets, and minimap layout rules.	A seeded run creates a playable branch of varied room templates, supports traversal/revisit state, and remains save/load compatible.
M15	Item, Card, and Reward Economy V1	Move deterministic rewards into ScriptableObject pools, reward choice UI, passive effect stack rules, active item hooks, shop/reward room scaffolding, and balance tuning sheets.	Runs can grant multiple item/card effects from pools, display choices, save collected effects, and apply stats predictably.
M16	Enemy and Boss Content V1	Add multiple enemy brains, ranged/projectile enemies, spawn formations, boss encounter scripting, telegraphs, difficulty tiers, and combat readability QA tools.	A branch contains distinct encounters and one boss-room prototype with readable attacks, death, reward, and save/load behavior.
M17	Player Feel, HUD, and Accessibility Pass	Improve movement/aim feel, hit feedback, health/readability, controller and keyboard rebinding, pause/death/retry UI, and platform-specific HUD comfort.	The game feels intentionally playable on Windows and Vision Pro modes, with clear feedback and no required developer knowledge.
M18	Persistence Hardening and Migration	Add save backups, schema migration tests beyond v2, corruption recovery, explicit transient session audit hooks, and optional telemetry-friendly event logs.	Bad or old save files fail safely, profiles recover gracefully, and transient sessions remain provably non-mutating.
M19	Content Import and Art Replacement Pipeline	Define production model/material/audio import rules, Addressables build validation, prefab validators, LFS policy checks, and authored room visual replacement workflow.	Artists can drop assets into documented folders and get actionable validation errors before runtime.

Milestone	Goal	Implementation Focus	Acceptance Criteria
M20	Vertical Slice Content Lock	Assemble a short but complete run: several rooms, one boss, reward curve, menus, saves, platform builds, QA checklist, and handoff build artifacts.	A non-developer can start a profile, complete a prototype run, return to menu, and continue or start a new run on supported platform routes.

Suggested Immediate M12 Scope

M12 should be deliberately practical. The goal is not new gameplay; the goal is to make the prototype reproducible by another engineer and to turn M11 from a local validation pass into a handoff pipeline.

- Add Hollow/Validation/Run Full Prototype Audit to execute M0-M11 validators and write JSON/Markdown output.
- Add build scripts or editor menu items for Windows development builds and placeholder visionOS build profile checks.
- Create output/builds and output/reports conventions with versioned artifact names.
- Add build-number stamping and a generated build manifest that records Unity version, git branch/hash, scenes, Addressables profile, and validation status.
- Add a PlayMode boot smoke test for MainMenu -> New Run -> first room loaded.
- Document exact local build commands in README and Docs/Milestone12BuildAutomation.md.

Audit Conclusion

M0-M11 achieved the intended prototype lock. The architecture is recognizable against the original Unity plan, validators and tests pass, and the project has enough structure to continue safely. The biggest next decision is whether to prioritize production build reliability (recommended M12) or gameplay scope expansion. The healthier path is build reliability first, then macro rooms and procedural branch generation.